Dogzy sur la blockchain : guide simple

Copie du travail de GitHub Copilot, prise le 13 octobre 2026. Source : dépôt `emm2026-tera/valkyrians`, branche `copilot/create-smart-contract-export-dogzy-tokens`, version (commit) `7cc6717`, datée du 13 oct. 2026 à 8 h 58 (heure de Toronto). **Attention : Copilot travaillait peut-être encore sur cette branche. Ce travail n'est pas encore intégré au projet principal.**

Comment ça marche, en clair

1. Dans le jeu, le joueur a des jetons Dogzy virtuels.
2. Il demande à les échanger. Le serveur du jeu retire ces jetons de son compte dans le jeu.
3. Le serveur lui remet un « bon » signé numériquement (comme un chèque signé). Ce bon ne sert qu'une seule fois, et seulement pour ce joueur. Il expire après un certain délai.
4. Le joueur présente ce bon au contrat « pont » (DogzyBridge) sur Avalanche, depuis son portefeuille crypto. Il reçoit alors de vrais jetons DOGZY.
5. Le pont paie à partir d'une réserve de DOGZY que tu y déposes, toi, la propriétaire.

Comme propriétaire, tu peux mettre le pont en pause, changer le taux d'échange, changer la clé de signature du serveur, plafonner les montants, et retirer la réserve.

Les dossiers

- **01-contrats** : les programmes qui vivent sur la blockchain.
- `DogzyToken.sol` crée le jeton « Dogzy » (symbole DOGZY). Par défaut, il y en a 1 milliard, tous donnés à la personne qui l'installe.
- `DogzyBridge.sol` est le pont qui vérifie les bons et paie les joueurs.
- **02-scripts-deploiement** : `deploy.js` installe les deux contrats. Par défaut il les installe sur Fuji (le test gratuit). Il refuse le vrai réseau tant qu'on n'a pas confirmé exprès (`CONFIRM_MAINNET=yes`).
- **03-tests** : `DogzyBridge.security.js` contient 12 tests de sécurité. Ils vérifient qu'un bon ne peut pas servir deux fois ni être volé, et qu'un faux bon, un montant modifié ou un bon expiré sont refusés. Ils vérifient aussi le plafond, la réserve, la pause et que les commandes réservées à la propriétaire sont bien protégées.
- **04-configuration** : les réglages techniques.
- `hardhat.config.js` donne les adresses des réseaux Avalanche.
- `package.json` et `package-lock.json` listent les outils nécessaires.
- `.env.example` est le modèle des secrets à remplir.
- `.gitignore` empêche d'envoyer les secrets sur GitHub.
- **05-documentation** : `README.md` contient les explications techniques de Copilot, en anglais et en français.

Ce qui est fait

- Les deux contrats et le script d'installation sont écrits.
- **Vérifié sur l'ordinateur le 13 oct. 2026 :** la compilation (`npx hardhat compile`) réussit, et **les 12 tests de sécurité passent, 0 échec**.

Ce qui reste

- **Rien n'est installé sur la blockchain pour l'instant.**
- Le serveur du jeu qui retire les jetons et signe les bons n'existe pas encore. Il n'est pas dans ce dossier.
- Le README parle d'un dossier `contracts/mocks`. Ce dossier n'existe pas dans le dépôt, et les tests n'en ont pas besoin.

Prochaines étapes

1. **Test sur Fuji (gratuit)** : obtenir des AVAX de test gratuits, remplir `.env`, puis lancer le script avec `--network fuji`.
2. **Vrai réseau Avalanche** : il faut un portefeuille (Core ou MetaMask) et un peu d'AVAX. Ça coûte en général de quelques cents à quelques dollars. Une fois installé, un contrat ne peut plus être retiré.
3. **Secrets** : la clé du portefeuille d'installation et la clé de signature du serveur ne doivent jamais être partagées, mises sur GitHub ou écrites dans un chat. Quiconque a la clé de signature peut créer des bons.
4. **Audit de sécurité professionnel** avant d'y mettre de la vraie valeur. Les tests automatiques ne remplacent pas un audit.